Technical Specification

FinTech Requirement

Table of contents

1. General Consideration

1. Since this application is designed to serve over 5 million active users, I have prioritized infrastructure planning with a strong focus on availability, security, and performance to meet the client's key objectives. Infrastructure cost will be optimized but not limited.

2. Currently, I have planned the deployment on GCP, based on my experience, but the application is designed to be cloud-agnostic, allowing deployment on other cloud platforms with minimal or no modifications.

Note:

I haven't implemented any code, due to time constraints but tried to elaborate at architecture side as much as possible.

2. Top level architecture

1. Microservices

- Various microservices can be designed to handle specific tasks and responsibilities within the system. Based on my initial assessment, the following services could be a suitable starting point for development.
- Authentication & Authorization Service
- Account Management Service
- Bill Payment Service
- Transaction Processing Service
- Payment Gateway Integration Service
- Notification & Alerts Service
- Reporting & Analytics Service
- Caching & Session Management Service
- Fraud Detection & Risk Analysis Service
- Few more as per requirement

Microservices within a Kubernetes cluster can communicate with each other using DNS-based service discovery. For example, a service like account-service can be accessed using its fully qualified domain name (FQDN) such as account-service.finance-system-dev.svc.cluster.local. This approach ensures seamless internal communication while maintaining service discoverability and load balancing.

To expose microservices outside the cluster, an **Ingress Controller** can be used. It acts as an entry point, managing external access through HTTP/HTTPS routes and providing features like load balancing, SSL termination, and routing based on hostnames or paths. Popular ingress controllers include **NGINX Ingress Controller**, **Traefik**, and **AWS Application Load Balancer** for cloud environments.

- Additionally, for scenarios requiring direct external access without complex routing, services can be exposed using a **LoadBalancer** service type or **NodePort**. However, using an ingress controller is generally preferred for managing multiple services with a single external endpoint.

  Implementing proper network policies and TLS encryption is essential to ensure secure communication between services, both internally and externally. Monitoring and logging tools like **Prometheus** and **Grafana** can also be integrated to observe service health and troubleshoot connectivity issues efficiently

.

2. Security, External communication with DBs/API/ System consideration

1. dedicated VPC has to be created to isolate resources and for the better control on security.
2. Dedicated service account with least privilege has to be created for separate environments.
3. To ensure consistent standards and maintain security across the cloud environment, organization-level policies should be established and enforced. These policies serve as a governance framework, defining best practices, access controls, data management guidelines, and compliance requirements. By implementing policies at the organizational level, businesses can minimize security risks, prevent misconfigurations, and ensure regulatory compliance. Additionally, these policies provide clear accountability and streamline cloud operations, promoting a secure and well-managed infrastructure. Regular audits and automated policy enforcement can further enhance visibility and control across all cloud resources.
4. For financial system best and secure way to communicate with external DB/System/Api is through VPN/VPC/tunnels or private network.
5. For external Database, secure VPC tunnels can be established to connect.
6. Production server has to be behind the WAF to protect against DDOs attacks and bots.

7. All cloud services provide native support for default encryption and security. However, using a customer-managed key (CMK) can further enhance security by giving you greater control over encryption management.

3. Technology selection and justifications

Tech Stack	Remarks/Justification
Cloud hosting	
Kubernetes (GKE/EKS/AKS)	To host the microservices and front-end **Reason:** Kubernetes serves as an ideal platform for hosting large systems due to its advanced container orchestration capabilities, ensuring seamless scalability, high availability, and operational resilience. It provides automated workload management, dynamically adjusting resources based on demand, which is critical for financial applications experiencing variable traffic patterns. Kubernetes also enhances fault tolerance through self-healing mechanisms, automatically detecting and recovering from container or node failures while maintaining service continuity. Its robust security framework, including Role-Based Access Control (RBAC), network policies, and workload isolation, ensures compliance with stringent financial regulations and data protection standards. Features like rolling updates, blue-green deployments allow seamless application updates with minimal downtime, maintaining consistent service availability. Furthermore, Kubernetes' native observability tools facilitate real-time monitoring, logging, and tracing, providing actionable insights into system performance and ensuring proactive issue resolution. By leveraging Kubernetes, financial systems can achieve enterprise-grade reliability, operational efficiency, and compliance, while accelerating innovation through agile application delivery

PostgreSQL	Database **Reason:** PostgreSQL is an excellent choice for our financial system due to its robust support for **ACID-compliant transactions**, ensuring **data integrity** and **consistency** — critical for financial operations. It provides powerful capabilities for **horizontal read scaling** by incorporating **read-only replicas**, which can effectively handle increased read workloads, particularly during high-traffic periods. Furthermore, with the use of extensions like **pgvector**, PostgreSQL supports **vector and embedding storage**, enabling seamless integration with **AI-driven solutions** for **fraud detection**, **risk management**, and **personalized financial insights** using existing data. This makes it a **future-ready choice** as we expand AI applications within our platform. For scenarios demanding enhanced **performance** and **real-time analytics** on large datasets, **AlloyDB** can serve as an optimal solution. It provides PostgreSQL compatibility while offering superior query performance and accelerated analytical capabilities, ensuring faster decision-making and improved customer experiences. If the requirement includes **cross-region support** with seamless data consistency and high availability, **PostgreSQL** does not provide native cross-region replication. While external tools and managed solutions like **Cloud SQL** offer read replicas and failover options, they may introduce complexities in maintaining global consistency. In contrast, **Cloud Spanner** is purpose-built for globally distributed databases, offering **strong consistency**, **horizontal scaling**, and **99.999% uptime** across regions, making it an ideal choice for large-scale financial systems requiring real-time, multi-region data management. Cloud Spanner does support for vector search

	--- However, if regulatory requirements such as **GDPR** or other data sovereignty laws mandate that data remains within specific regions, **PostgreSQL** becomes a more appropriate choice. It allows strict control over data locality, ensuring compliance by keeping data within designated regions. Additionally, its robust support for **ACID transactions**, data security, and flexible deployment options make it a reliable solution for region-specific financial applications.
React.js with **Next.js**	For frontend applications.
.Net core OR Java OR Node	For backend/ APIs
Python	(For AI-driven fraud detection & analytics)
SonarCloud (cloud hosted) / SonarQube (Self hosted)	To ensure quality and maintainability Detect Security Vulnerabilities Regulatory Compliance Can be used with GitHub action to validate code before deploy. Note: ESLint can also be considered for local development for limited to JavaScript/TypeScript
Pub-sub and Cloud functions	use of **Google Cloud Pub/Sub**, **Cloud Functions**, and **Cloud Schedulers** to manage backend jobs for financial statement generation, email messaging, and similar asynchronous tasks. This serverless, event-driven architecture ensures reliability, scalability, and cost-

	efficiency for processing large volumes of backend operations.
Load balancer	
GCP buckets	To store assets and user statements and other related documents securely and cost-effective way.
GitHub	Source controller For this application, a Turborepo-based approach can be considered for script-based development, enabling efficient monorepo management and shared dependencies. For a Java-based application, a separate repository per service can be created and integrated with Jenkins jobs for build and deployment, ensuring clear separation and streamlined CI/CD.
DevOps	Continues Development and Deployment GitHub - Facilitates **collaborative development** with pull requests, code reviews, and branching strategies. - Integrates seamlessly with CI/CD tools for automated builds and deployments. - Provides **GitHub Actions** for native CI/CD workflows. Jenkins J*enkins is an open-source automation server for continuous integration and delivery.* • Highly customizable with **thousands of plugins** for integration with various tools. • Supports **pipeline as code** using Jenkins file for defining CI/CD workflows. • Facilitates **parallel and distributed builds** for faster processing.

	• Ideal for managing complex, multi-stage pipelines across hybrid environments. Spinnaker - Open source for CD - Supports **multi-cloud deployments** across GCP, AWS, and other providers. - Enables **progressive rollouts** with blue/green and rolling updates. - Allow different trigger/repo integration for automatic pipeline triggers like pub-sub, repo submit etc. - https://netflixtechblog.com/global-continuous-delivery-with-spinnaker-2a6896c23ba7 Cloudbuild - - Supports **parallel execution** of build steps to reduce build time. - Provides secure **container image builds** and storage with Artifact Registry. - Simplifies pipeline management using **Cloud Build YAML configuration**. Skaffold - Pipeline Management: - Container Image Management: supports multiple image builders like Docker, Buildpacks, and Kaniko. - **CI/CD Integration:** Compatible with Jenkins, Cloud Build, and GitHub Actions for automated pipelines - **Consistency Across Environments:**

	Ensures that applications behave the same across local, staging, and production environments - Terraform: - **Infrastructure Automation:** Automate the provisioning and management of cloud infrastructure like **VM**, **Kubernetes clusters**, **storage buckets**, and **databases**. - https://registry.terraform.io/ - Create consistence infrastructure across environments
SRE (Site Reliability Engineering) tools:	Prometheus Grafana Slack/Team Cloud monitoring configuration
Web Application Firewall (WAF)	Load balancer or any third party WAF

4. Cost optimization consideration

1. **Leverage Committed Use Discounts:** Purchase committed use discounts for compute instances that require continuous uptime to significantly reduce costs.
2. **Optimize Dev and Test Environments:** Implement automated scheduling to scale down development and testing environments during off-hours or weekends, leading to substantial cost savings.
3. **Utilize Serverless Services:** Where applicable, adopt serverless solutions that offer pay-as-you-go pricing and often include a free tier, reducing operational expenses.
4. **Embrace Open Source Solutions:** Use open-source tools like Spinnaker, Jenkins, and Grafana to minimize software licensing expenses.

5. **Implement Resource Monitoring and Alerts:** Use cloud-native monitoring tools like GCP's Cloud Monitoring and Cloud Logging to track resource usage and set up alerts to detect anomalies early.
6. **Adopt Auto-Scaling and Right-Sizing:** Configure auto-scaling for applications to dynamically adjust resources based on demand, and periodically review instance sizes to ensure they align with actual usage.
7. **Use Spot and Preemptible Instances:** For workloads with flexible time requirements, consider using spot or preemptible instances for cost-effective computing. This can be done for dev and test clusters. Also, for the process like batch operations, migration activities etc.
8. **Optimize Data Storage:** Choose cost-efficient storage classes like Nearline or Coldline for infrequently accessed data and enable lifecycle management policies to automatically archive or delete obsolete data.
9. **Network Cost Management:** Minimize data transfer costs by using regional resources and implementing Content Delivery Networks (CDNs) where appropriate.

1.2 Objectives

- Provide seamless and secure online banking services to over 5 million active users.

<My comments>

To provide seamless and secure online banking services to over 5 million active users, a combination of appropriately sized and highly available cloud resources can be utilized.

A **Kubernetes** cluster with **Horizontal Pod Autoscaler (HPA)** can ensure the application scales automatically based on traffic demands, maintaining performance and reliability. This enables dynamic scaling during peak hours and reduces infrastructure costs during low-traffic periods. Implementing **multi-zone** or **multi-region** deployments further enhances fault tolerance and uptime.

For event-driven workflows and background tasks, **Pub/Sub** can be leveraged to manage asynchronous messaging and decouple services. Combined with **Cloud Functions** or **Cloud Run**, it provides an efficient, serverless solution for processing events in real-time, such as transaction notifications, fraud detection, or account updates.

Additionally, using **Managed Databases** like Cloud Spanner or Cloud SQL can ensure low-latency, highly available data storage with global replication. Implementing **Redis** for caching frequently accessed data can further optimize performance.

To enhance security, **Identity and Access Management (IAM)**, **Cloud Armor** for DDoS protection, and **Cloud Logging and Monitoring** for real-time observability are essential. Applying encryption for data at rest and in transit, along with role-based access controls, ensures compliance with banking regulations.

This approach provides a scalable, resilient, and secure infrastructure, delivering a reliable online banking experience to millions of users.

- Ensure real-time transaction processing while handling traffic spikes (e.g., payroll days, shopping seasons).

  <My Comments>

- To ensure real-time transaction processing while handling traffic spikes (e.g., payroll days or shopping seasons), an auto-scaling approach can be implemented. In a Kubernetes environment, Horizontal Pod Autoscaler (HPA) can be configured for each microservice to dynamically scale the number of pods based on CPU or memory usage. Alternatively, for virtual machine-based deployments, multiple instances can be automatically spun up using instance templates and managed instance groups (MIGs) to match the demand. Both approaches ensure optimal resource utilization and maintain service availability during peak loads

- Ensure compliance with financial regulations (GDPR, PCI DSS, PSD2, SOC 2).

  This needs to consider at both the levels, development and hosting/infrastructure level.

  All data needs to encrypt at rest using key management or encryption service. For more control, client managed keys can also be used.

  Data masking technique can be used to store sensitive data.

  Enforce SSL/TLS connections to encrypt data in transit. Configure DB service in such a way to only accept secure connection and restrict access to internal access.

  Enable detail monitoring/logging of network/access/connections.

- Build a modular, microservices-based architecture for scalability and resilience.

To achieve scalability and resilience, a modular, microservices-based architecture will be implemented. This approach involves decomposing the application into independent, loosely coupled services, each responsible for a specific business capability.

- **Microservice Decomposition:**
  1. As outlined in section 2.1, the proposed architecture includes microservices such as, Authentication & Authorization Service, Account Management Service.
  2. Each microservice will be designed with a clear, well-defined API, enabling independent development, deployment, and scaling.
  3. Each microservice will be deployed in Kubernetes as a deployment.
- **Scalability:**
  1. Microservices enable horizontal scalability. Individual services can be scaled independently based on their specific demand. For example, the **email-statement** service can be scaled during beginning of the month for send emails or such monthly processes/tasks.
  2. This granular scaling optimizes resource utilization and reduces costs.

- Implement AI-driven fraud detection, risk management, and transaction monitoring.

  <My Comments>

  To enhance fraud detection capabilities, we can explore multiple approaches using AI. One option is to leverage third-party services specializing in fraud detection, which often provide pre-trained models with real-time monitoring and anomaly detection. These services can be quickly integrated into existing systems, reducing implementation time and effort. Enable smooth third-party service integrations (payment networks, banking APIs, forex, investment services.

  Alternatively, we can opt for a more customized solution by downloading open-source Large Language Models (LLMs) or Small Language Models (SLMs). Using Python libraries such as TensorFlow, python transformer, PyTorch, or Hugging Face, these models can be fine-tuned with our own data to detect fraudulent patterns specific to the business. This approach offers greater flexibility and control over the model's behavior and decision-making but another side, it will need higher deployment resources and also expertise.

2. Functional Requirements

2.1 User Account Management

- Secure user registration and onboarding with Know Your Customer (KYC) verification.
- Multi-factor authentication (MFA) for login, transfers, and high-risk actions.
- User profile and preferences management.
- Role-based access control (RBAC) for different user roles (e.g., customers, admins, compliance officers).

2.2 Transactions & Payments

- Fund transfers between customer accounts.
- Bill payments with automated scheduling.
- International wire transfers with currency conversion.
- Recurring payments and subscriptions.
- Real-time transaction processing with instant balance updates.
- Fraud prevention mechanisms (e.g., transaction velocity checks, anomaly detection).

2.3 Security & Compliance

- End-to-end encryption (TLS 1.3 for data in transit, AES-256 for data at rest).
- PCI DSS compliance for handling credit/debit card transactions.
- GDPR compliance for European users' personal data storage and processing.
- Audit logging for all financial activities.
- User consent management for privacy and regulatory needs.

2.4 Banking Services & Financial Analytics

- Checking and savings account management.
- Investment products (e.g., ETFs, bonds, mutual funds) integration.
- Loan and credit services (eligibility, applications, repayments).
- AI-powered spending analytics and financial planning insights.

2.5 Notifications & Alerts

- Real-time push notifications, SMS, and email alerts for transactions, security warnings, and promotions.
- Alert users on unusual spending patterns and potential fraud.
- Monthly account statements via email or secure document access.

2.6 Customer Support & Dispute Resolution

- AI chatbot and live agent support for assistance.
- Dispute resolution workflow for unauthorized transactions or errors.

<complain/solution> module can be implemented to achieve this>.

- Secure messaging platform for customer-bank interactions.

to build a **secure messaging platform for customer-bank interactions**, an **AI-powered conversational system** can be implemented to enhance customer service and streamline communication.

A **Conversational AI** solution, powered by **Google Dialogflow CX**, **Azure Bot Service**, or **OpenAI GPT models**, can be integrated to provide an intelligent chatbot capable of handling inquiries, processing transactions, and assisting users with banking services. By leveraging **Natural Language Processing (NLP)** and **Machine Learning (ML)**, the chatbot can offer personalized responses and improve over time based on user interactions.

3. Non-Functional Requirements

3.1 Scalability & Performance

- Microservices-based architecture ensuring modularity, independent scaling, and fault isolation.
- Auto-scaling cloud infrastructure to handle high transaction volumes.
- Distributed databases with sharding and read replicas for performance.
- Content Delivery Network (CDN) for low-latency access to static assets.

<Static assets can be stored in storage bucket (which is cost optimized, high available) which can be exposed through CDN for better performance>

3.2 Security & Fraud Prevention

- Zero-trust architecture ensuring strict authentication for all services.

< To implement a **Zero-Trust Architecture (ZTA)** ensuring strict authentication for all services, a **deny-by-default** approach can be adopted. This principle, often the default configuration in most firewalls, ensures that all incoming and outgoing traffic is blocked unless explicitly permitted. Only the necessary ports, protocols, and target services should be allowed based on well-defined security policies.

In addition to firewall rules, **identity-based access controls** can be enforced using solutions like **Identity-Aware Proxy (IAP)** or **Cloud IAM**. This ensures that users and services are authenticated and authorized before accessing resources. Implementing **mutual TLS (mTLS)** for service-to-service communication within a Kubernetes environment further strengthens authentication and encryption.

Network segmentation using **Virtual Private Cloud (VPC)** with **private subnets** and **firewall rules** can minimize the attack surface. Additionally, employing **service mesh** solutions like **Istio** can provide granular access controls, traffic management, and observability.

Continuous monitoring using tools like **Cloud Audit Logs** and or WAF monitoring can detect and respond to suspicious activities in real time. Implementing **zero-trust principles** like least privilege access, continuous verification, and end-to-end encryption ensures a highly secure infrastructure.>

- DDoS protection and Web Application Firewall (WAF) to prevent cyberattacks.

<Any third party WAF or inbuild WAF can be used, production has to be behind the WAF>

- Behavioral analytics and AI-driven fraud detection to flag suspicious activities.
- Rate limiting & transaction throttling to prevent abuse.
- API security with OAuth 2.0 and JWT for authentication and authorization.

3.3 Compliance & Data Privacy

- GDPR-compliant data residency policies ensuring sensitive data remains in respective regions.

To ensure GDPR-compliant data residency policies, global or organization-level cloud policies should be established and enforced. These policies can define strict rules governing resource creation, data storage, and cross-service account sharing to ensure compliance with regional regulations.

For GDPR compliance, it is essential to mandate that all sensitive data remains within the **EU region**. Cloud providers offer features like **resource location restrictions** and **data residency controls** to ensure data is stored and processed in designated regions. Implementing **Organization Policies** in platforms like **Google Cloud** or **Azure Policy** in **Microsoft Azure** can prevent resources from being created outside of authorized locations.

Additionally, **encryption** should be applied for data at rest and in transit using **Customer-Managed Keys (CMKs)** for enhanced control. **Data Access Monitoring** and **Audit Logging** can ensure transparency and traceability of data usage.

.

- Tokenization of payment card details to reduce PCI DSS compliance burden.
- Strong audit logging for transaction history and system access tracking.
- Right to be forgotten and data portability features for compliance.

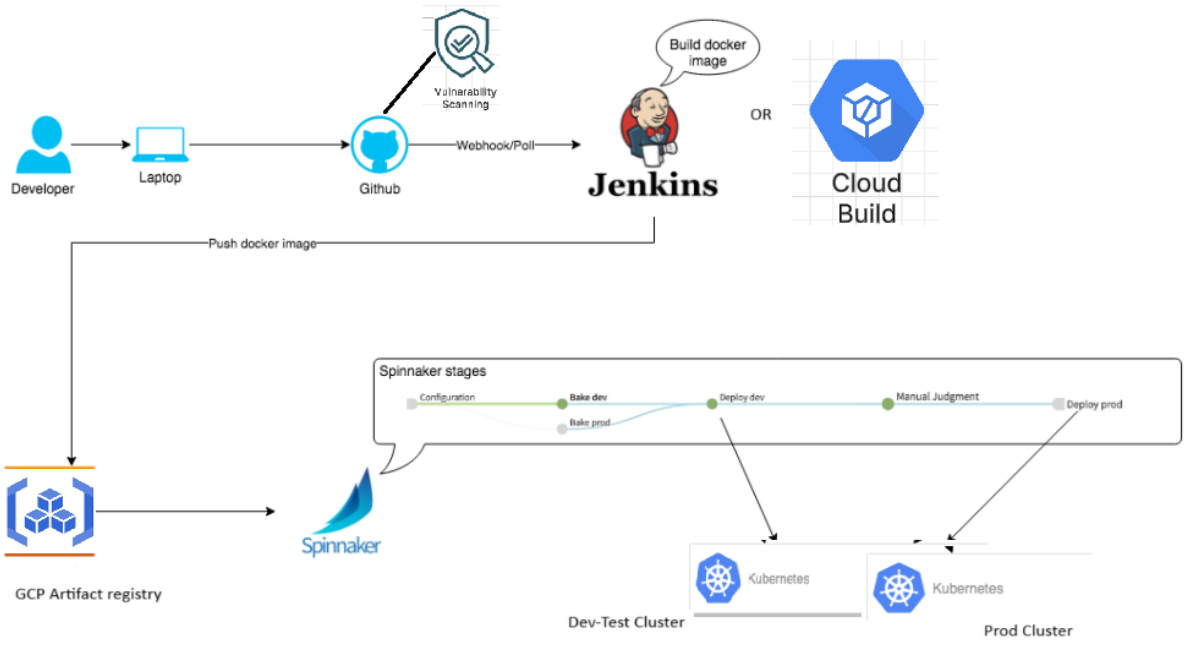
3.4 High Availability & Disaster Recovery

- Multi-region deployment (active-active or active-passive with failover strategies).
- Data replication and backup strategies ensuring zero data loss (RTO < 5 min, RPO < 1 min).
- Blue-green deployment strategy for zero-downtime updates.

3.5 Maintainability & Observability

- CI/CD pipelines for automated testing, vulnerability scanning, and code deployment.

<container registry scanning can be enabled by default, GitHub or SonarCloud can also be configured to scan the code vulnerability.>



- Real-time monitoring & alerting via ELK stack, Prometheus, or cloud monitoring solutions.

<This can be done with different monitoring tools, monitoring policies and configuration with alert tools like slack, team, email>

- Service mesh for observability and secure inter-service communication.
- Automated logging & tracing to detect performance bottlenecks and security threats.

<Prometheus with Grafana can be setup to keep eye on workload resource and alert can be generated in case of high load or unusual load.

Different tools can be used to monitor the user activities like user permission changes>

4. Third-Party Integrations

4.1 Payment & Banking APIs

- Stripe / PayPal / Plaid for payment processing and bank account linking.
- SWIFT / SEPA / ACH integrations for domestic and international transfers.
- Forex API for real-time exchange rates.

4.2 Messaging & Notifications

- Twilio / SendGrid / AWS SES for SMS, push, and email notifications.
- WhatsApp Business API for customer support.

4.3 Compliance & Fraud Detection

- SOC 2 certified monitoring tools for continuous compliance.

- AI-driven fraud detection engine with real-time alerts.
- AML (Anti-Money Laundering) checks via third-party APIs.